

**LAPORAN WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)**

SEMESTER III



BKPM: ACARA 8

**NAMA:
SHERYN FEBRYLIA HENDRITA
(E41250604)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

A. Tujuan

Menerapkan operasi CRUD (*Create, Read, Update, Delete*) pada resource mahasiswa, program studi, dan mata kuliah menggunakan PDO prepared statement. Selain itu, membuat database class dengan pola singleton, menerapkan JOIN untuk menampilkan nama program studi pada data mahasiswa, menggunakan Bootstrap untuk memperbaiki tampilan halaman, serta menambahkan fitur pencarian berdasarkan nama atau NIM.

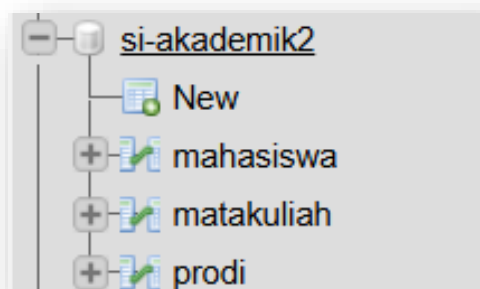
B. Studi Kasus

Sistem Informasi Akademik membutuhkan fitur pengelolaan data mahasiswa, program studi, dan mata kuliah. Pada Acara 8, aplikasi dikembangkan agar dapat melakukan proses menambah, menampilkan, mengubah, dan menghapus data.

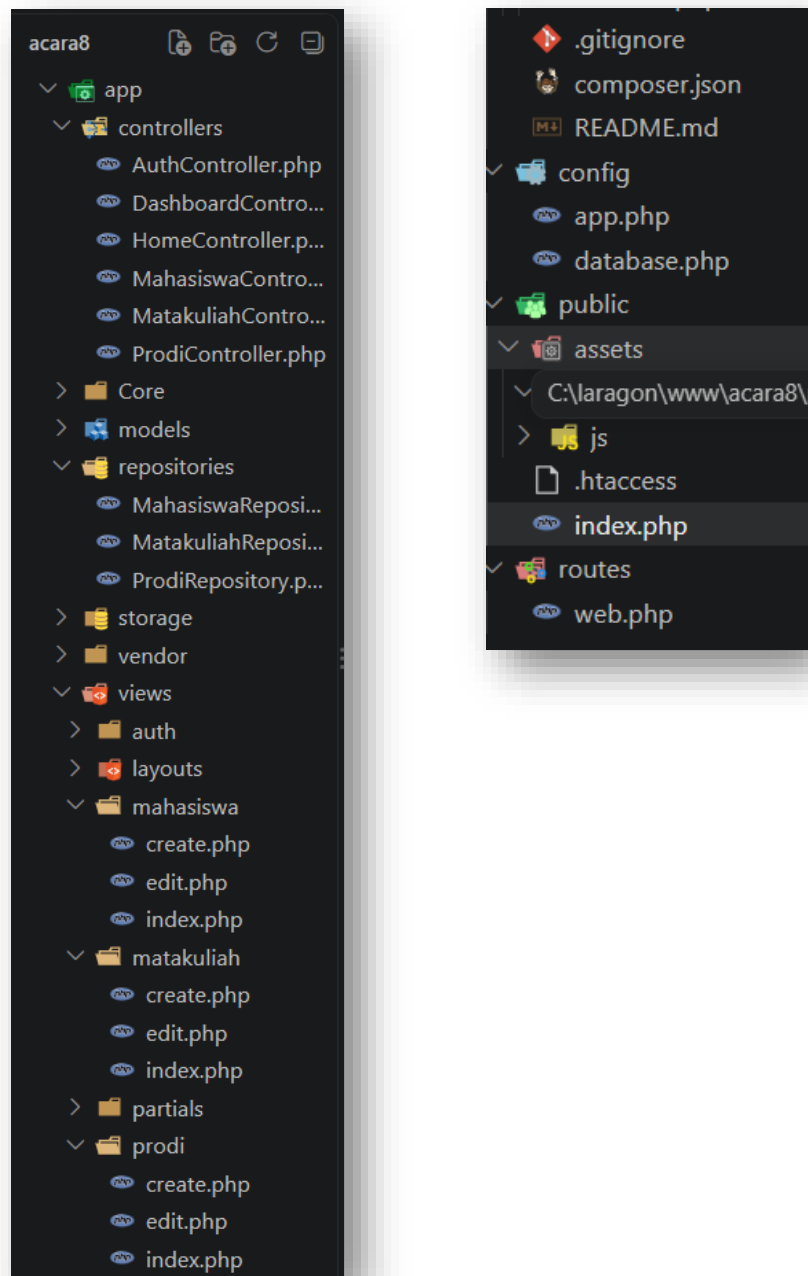
Penggunaan query yang langsung menggabungkan input pengguna ke dalam SQL dapat menyebabkan SQL Injection. Oleh karena itu, aplikasi menggunakan PDO prepared statement agar input pengguna dipisahkan dari perintah SQL.

Selain itu, digunakan JOIN untuk menampilkan nama program studi pada daftar mahasiswa dan Bootstrap untuk membuat tampilan aplikasi lebih rapi.

C. Struktur Database



D. Struktur Folder



E. Langkah Pengerjaan

1. Membuat Database Class Singleton dengan PDO app/Core/Database.php

```

app > Core > Database.php > Database
1  <?php
2
3  6 references | 0 implementations
4  class Database
5  {
6
7      3 references
8      private static ?PDO $instance = null;
9
10
11      3 references | 0 overrides
12      public static function getInstance(): PDO
13      {
14          if (self::$instance === null) {
15              $config = require __DIR__ . '/../../config/database.php';
16              $dsn = "mysql:host={$config['host']};dbname={$config['dbname']};charset={$config['charset']}";
17
18              self::$instance = new PDO(
19                  $dsn,
20                  $config['username'],
21                  $config['password'],
22                  [
23                      PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
24                      PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
25                      PDO::ATTR_EMULATE_PREPARES => false,
26                  ]
27              );
28          }
29          return self::$instance;
30      }
31  }

```

Penjelasan:

Class Database digunakan untuk membuat koneksi aplikasi dengan database MySQL menggunakan PDO. Pola singleton diterapkan dengan variabel static \$instance sehingga koneksi database dibuat satu kali dan dapat digunakan kembali. Pengaturan PDO::ATTR_EMULATE_PREPARES => false digunakan agar prepared statement PDO tidak menggunakan emulasi.

2. Membuat Repository Mahasiswa

app/repositories/MahasiswaRepository.php

all()

find()

create()

update()

delete()

- method all():

```
12 public function all(string $keyword = ''): array
13 {
14     if ($keyword !== '') {
15
16         $stmt = $this->pdo->prepare("
17         SELECT
18             m.*,
19             p.nama AS prodi_nama
20         FROM mahasiswa m
21         JOIN prodi p ON m.prodi_id = p.id
22         WHERE m.nim LIKE :nim
23             OR m.nama LIKE :nama
24         ORDER BY m.nim
25         ");
26
27         $search = '%' . $keyword . '%';
28
29         $stmt->execute([
30             'nim' => $search,
31             'nama' => $search
32         ]);
33
34         return $stmt->fetchAll();
35     }
36
37     $stmt = $this->pdo->query("
38     SELECT
39         m.*,
40         p.nama AS prodi_nama
41     FROM mahasiswa m
42     JOIN prodi p ON m.prodi_id = p.id
43     ORDER BY m.nim
44     ");
45
46     return $stmt->fetchAll();
47 }
```

Penjelasan:

Method all() digunakan untuk mengambil seluruh data mahasiswa. Query menggunakan JOIN dengan tabel prodi sehingga nama program studi dapat ditampilkan. Method ini juga menerima parameter \$keyword untuk melakukan pencarian berdasarkan NIM atau nama.

- Method find()

```

49     public function find(int $id): ?array
50     {
51         $stmt = $this->pdo->prepare(
52             "SELECT * FROM mahasiswa WHERE id = :id"
53         );
54
55         $stmt->execute([
56             'id' => $id
57         ]);
58
59         $data = $stmt->fetch();
60
61         return $data ?: null;
62     }

```

Penjelasan:

Method `find()` digunakan untuk mengambil satu data mahasiswa berdasarkan ID. Query menggunakan `prepare()` dan `execute()` sehingga nilai ID dipisahkan dari perintah SQL.

- **Method `create()`**

```

64     public function create(array $data): int
65     {
66         $stmt = $this->pdo->prepare("
67             INSERT INTO mahasiswa
68             (nim, nama, email, prodi_id, angkatan)
69             VALUES
70             (:nim, :nama, :email, :prodi_id, :angkatan)
71         ");
72
73         $stmt->execute([
74             'nim' => $data['nim'],
75             'nama' => $data['nama'],
76             'email' => $data['email'],
77             'prodi_id' => $data['prodi_id'],
78             'angkatan' => $data['angkatan']
79         ]);
80
81         return (int) $this->pdo->lastInsertId();
82     }
83

```

Penjelasan:

Method `create()` digunakan untuk menambahkan data mahasiswa baru ke dalam database. Data dikirim menggunakan parameter prepared statement sehingga input pengguna tidak langsung digabungkan dengan query SQL.

- **Method `update()`**

```
84 public function update(int $id, array $data): bool
85 {
86     $stmt = $this->pdo->prepare("
87         UPDATE mahasiswa
88         SET
89             nim = :nim,
90             nama = :nama,
91             email = :email,
92             prodi_id = :prodi_id,
93             angkatan = :angkatan
94         WHERE id = :id
95     ");
96
97     return $stmt->execute([
98         'id' => $id,
99         'nim' => $data['nim'],
100        'nama' => $data['nama'],
101        'email' => $data['email'],
102        'prodi_id' => $data['prodi_id'],
103        'angkatan' => $data['angkatan']
104    ]);
105 }
106
```

Penjelasan:

Method `update()` digunakan untuk mengubah data mahasiswa berdasarkan ID. Data baru dikirim menggunakan prepared statement dengan parameter yang telah ditentukan.

- **Method `delete()`**

```

107     public function delete(int $id): bool
108     {
109         $stmt = $this->pdo->prepare(
110             "DELETE FROM mahasiswa WHERE id = :id"
111         );
112
113         return $stmt->execute([
114             'id' => $id
115         ]);
116     }
117 }

```

Penjelasan:

Method `delete()` digunakan untuk menghapus data mahasiswa berdasarkan ID. Prepared statement digunakan untuk memproses ID yang akan dihapus.

3. Membuat CRUD Mahasiswa pada Controller

`app/controllers/MahasiswaController.php`

```

11     public function __construct()
12     {
13         $pdo = Database::getInstance();
14         $this->repository = new MahasiswaRepository($pdo);
15     }
16

```

Penjelasan:

Constructor mengambil koneksi database melalui `Database::getInstance()`. Koneksi tersebut kemudian diberikan kepada `MahasiswaRepository` untuk digunakan dalam proses CRUD.

- **Menampilkan data mahasiswa**

```

18     public function index(): void
19     {
20         $keyword = trim($_GET['search'] ?? '');
21
22         $mahasiswa = $this->repository->all($keyword);
23
24         $this->view('mahasiswa/index', [
25             'mahasiswa' => $mahasiswa,
26             'keyword' => $keyword
27         ]);
28     }

```

Penjelasan:

Method `index()` digunakan untuk menampilkan data mahasiswa. Jika pengguna memasukkan kata kunci pencarian, kata tersebut dikirim ke Repository. Hasilnya kemudian dikirim ke View `mahasiswa/index`.

- **Menambah mahasiswa**

```

37     public function store(): void
38     {
39         $data = [
40             'nim' => trim($_POST['nim'] ?? ''),
41             'nama' => trim($_POST['nama'] ?? ''),
42             'email' => trim($_POST['email'] ?? ''),
43             'prodi_id' => (int) ($_POST['prodi_id'] ?? 0),
44             'angkatan' => (int) ($_POST['angkatan'] ?? 0)
45         ];
46
47         $this->repository->create($data);
48
49         $this->redirect('/acara8/public/mahasiswa');
50     }

```

Penjelasan:

Method `store()` mengambil data dari form menggunakan POST, kemudian mengirimkannya ke method `create()` pada Repository. Setelah data berhasil disimpan, pengguna diarahkan kembali ke halaman mahasiswa.

- **Mengubah mahasiswa**

```
69     public function update(int $id): void
70     {
71         $data = [
72             'nim' => trim($_POST['nim'] ?? ''),
73             'nama' => trim($_POST['nama'] ?? ''),
74             'email' => trim($_POST['email'] ?? ''),
75             'prodi_id' => (int) ($_POST['prodi_id'] ?? 0),
76             'angkatan' => (int) ($_POST['angkatan'] ?? 0)
77         ];
78
79         $this->repository->update($id, $data);
80
81         $this->redirect('/acara8/public/mahasiswa');
82     }
```

Penjelasan:

Method update() digunakan untuk menerima data hasil perubahan dari form kemudian mengirimkannya ke Repository berdasarkan ID mahasiswa.

- **Menghapus mahasiswa**

```
85     public function destroy(int $id): void
86     {
87         $this->repository->delete($id);
88
89         $this->redirect('/acara8/public/mahasiswa');
90     }
91 }
```

Penjelasan:

Method destroy() digunakan untuk menghapus data mahasiswa dengan memanggil method delete() pada Repository.

4. Membuat View Data Mahasiswa dengan Bootstrap

app/views/mahasiswa/index.php

Bootstrap digunakan dengan kode:

```
app > views > mahasiswa > index.php
1  <!DOCTYPE html>
2  <html lang="id">
3
4  <head>
5    <meta charset="UTF-8">
6    <meta name="viewport" content="width=device-width, initial-scale=1.0">
7    <title>Data Mahasiswa</title>
8
9    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
10 </head>
11
```

Data mahasiswa ditampilkan menggunakan tabel Bootstrap:

```
51 <table class="table table-bordered table-striped">
52
53   <thead class="table-dark">
54     <tr>
55       <th>No</th>
56       <th>NIM</th>
57       <th>Nama</th>
58       <th>Email</th>
59       <th>Prodi</th>
60       <th>Angkatan</th>
61       <th>Aksi</th>
62     </tr>
63   </thead>
64
```

Penjelasan:

Bootstrap digunakan untuk membuat tampilan halaman menjadi lebih rapi. Class table, table-bordered, dan table-striped digunakan untuk memberikan format pada tabel, sedangkan table-dark digunakan pada bagian header.

5. Menggunakan JOIN untuk Menampilkan Nama Prodi

```
38 SELECT
39   m.*,
40   p.nama AS prodi_nama
41 FROM mahasiswa m
42 JOIN prodi p ON m.prodi_id = p.id
43 ORDER BY m.nim
44 ");
```

Penjelasan:

Query tersebut menggabungkan tabel mahasiswa dengan tabel prodi menggunakan kolom prodi_id. Dengan JOIN, aplikasi dapat menampilkan nama program studi berdasarkan data relasi, bukan hanya menampilkan ID prodi.

Pada View digunakan:

```
90 <td><?= htmlspecialchars($m['prodi_nama']) ?></td>  
91
```

6. Membuat CRUD Program Studi

File Repository:

app/repositories/ProdiRepository.php

Repository program studi memiliki method:

all()

find()

create()

update()

delete()

Contoh method create():

```
43 public function create(array $data): int  
44 {  
45     $stmt = $this->pdo->prepare("  
46         INSERT INTO prodi  
47         (kode, nama)  
48         VALUES  
49         (:kode, :nama)  
50     ");  
51  
52     $stmt->execute([  
53         'kode' => $data['kode'],  
54         'nama' => $data['nama']  
55     ]);  
56  
57     return (int) $this->pdo->lastInsertId();  
58 }
```

Penjelasan:

Method `create()` digunakan untuk menambahkan data program studi. Data kode dan nama prodi dikirim menggunakan prepared statement.

Repository juga menyediakan method `find()`, `update()`, dan `delete()` untuk mengambil, mengubah, dan menghapus data prodi.

7. Membuat CRUD Mata Kuliah

File Repository:

`app/repositories/MatakuliahRepository.php`

Repository mata kuliah juga memiliki method:

`all()`

`find()`

`create()`

`update()`

`delete()`

Pada method `all()` digunakan JOIN:

```
14      $stmt = $this->pdo->query("
15          SELECT
16              m.*,
17              p.nama AS prodi_nama
18          FROM matakuliah m
19          JOIN prodi p ON m.prodi_id = p.id
20          ORDER BY m.kode
21      ");
22  
```

Penjelasan:

Method `all()` digunakan untuk menampilkan data mata kuliah sekaligus nama program studi. JOIN dilakukan antara tabel mata kuliah dan prodi berdasarkan `prodi_id`.

Untuk menambahkan data digunakan:

```
45      $stmt = $this->pdo->prepare("
46          INSERT INTO matakuliah
47          (kode, nama, sks, prodi_id)
48          VALUES
49          (:kode, :nama, :sks, :prodi_id)
50      ");

```

Prepared statement digunakan untuk memasukkan data mata kuliah secara aman.

7. Menampilkan data pada /mahasiswa

Pada View mahasiswa digunakan:

```
<form
  action="/acara8/public/mahasiswa/delete/<?= $m['id'] ?>"
  method="POST"
  style="display:inline;"
  onsubmit="return confirm('Yakin ingin menghapus data ini?');">

  <button type="submit" class="btn btn-danger btn-sm">
    Hapus
  </button>
</form>
```

Penjelasan:

Atribut onsubmit digunakan untuk menampilkan konfirmasi sebelum data dihapus. Jika pengguna memilih **Cancel**, proses penghapusan dibatalkan. Jika memilih **OK**, form dilanjutkan dan data diproses oleh Controller.

Konfirmasi yang sama juga diterapkan pada penghapusan data prodi dan mata kuliah.

E. Tugas Mandiri

Menambahkan Fitur Pencarian Mahasiswa

Pada tugas mandiri ditambahkan fitur pencarian mahasiswa berdasarkan **nama atau NIM**.

Pada Controller:

```
$keyword = trim($_GET['search'] ?? '');

$mahasiswa = $this->repository->all($keyword);
```

Penjelasan:

Controller mengambil kata kunci pencarian dari parameter search.

Kata kunci kemudian dikirim ke Repository untuk diproses.

Pada Repository digunakan prepared statement:

```
$stmt = $this->pdo->prepare("
SELECT
    m.*,
    p.nama AS prodi_nama
FROM mahasiswa m
JOIN prodi p ON m.prodi_id = p.id
WHERE m.nim LIKE :nim
    OR m.nama LIKE :nama
ORDER BY m.nim
");

$search = '%' . $keyword . '%';

$stmt->execute([
    'nim' => $search,
    'nama' => $search
]);
```

Penjelasan:

Query menggunakan LIKE untuk mencari data berdasarkan sebagian atau seluruh NIM dan nama mahasiswa. Tanda % digunakan agar pencarian dapat menemukan data yang mengandung kata kunci. Prepared statement digunakan agar input pencarian tidak langsung menjadi bagian dari perintah SQL.

Form pencarian pada View:

```

<!-- SEARCH -->
<form action="/acara8/public/mahasiswa" method="GET" class="mb-3">
  <div class="input-group">
    <input
      type="text"
      name="search"
      class="form-control"
      placeholder="Cari berdasarkan NIM atau nama..."
      value="<?= htmlspecialchars($keyword ?? '') ?>"
    >
    <button type="submit" class="btn btn-primary">
      Cari
    </button>
    <a
      href="/acara8/public/mahasiswa"
      class="btn btn-secondary">
      Reset
    </a>
  </div>
</form>

```

Penjelasan:

Form pencarian menggunakan metode GET dengan parameter search. Pengguna dapat memasukkan NIM atau nama mahasiswa kemudian menekan tombol **Cari**. Tombol **Reset** digunakan untuk kembali menampilkan seluruh data mahasiswa. sesuai tugas mandiri.

F. Hasil

Setelah seluruh langkah dilakukan, aplikasi berhasil menerapkan CRUD pada tiga resource, yaitu:

1. Mahasiswa
2. Program Studi
3. Mata Kuliah

Halaman mahasiswa dapat diakses melalui:

<http://localhost/acara8/public/mahasiswa>

localhost/acara8/public/mahasiswa

Data Mahasiswa

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari Reset

No	NIM	Nama	Email	Prodi	Angkatan	Aksi
1	2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	Edit Hapus
2	2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus
3	2401004	Sheryn Febrylia	e41250604@student.polije.ac.id	Teknik Informatika	2025	Edit Hapus
4	2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	Edit Hapus

Halaman program studi:

<http://localhost/acara8/public/prodi>

localhost/acara8/public/prodi

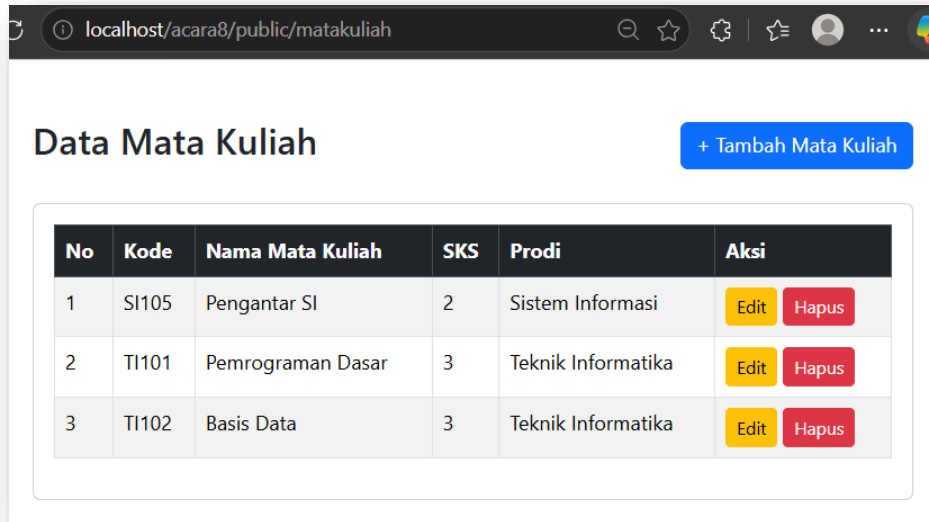
Data Prodi

+ Tambah Prodi

No	Kode	Nama Prodi	Aksi
1	SI	Sistem Informasi	Edit Hapus
2	TI	Teknik Informatika	Edit Hapus
3	TK	Teknik Komputer	Edit Hapus

Halaman mata kuliah:

<http://localhost/acara8/public/matakuliah>



localhost/acara8/public/matakuliah

Data Mata Kuliah

+ Tambah Mata Kuliah

No	Kode	Nama Mata Kuliah	SKS	Prodi	Aksi
1	SI105	Pengantar SI	2	Sistem Informasi	Edit Hapus
2	TI101	Pemrograman Dasar	3	Teknik Informatika	Edit Hapus
3	TI102	Basis Data	3	Teknik Informatika	Edit Hapus

Fitur yang berhasil diterapkan meliputi:

- Menampilkan data mahasiswa.
- Menambahkan data mahasiswa.
- Mengubah data mahasiswa.
- Menghapus data mahasiswa.
- Menampilkan data program studi.
- Menambahkan data program studi.
- Mengubah data program studi.
- Menghapus data program studi.
- Menampilkan data mata kuliah.
- Menambahkan data mata kuliah.
- Mengubah data mata kuliah.
- Menghapus data mata kuliah.
- Menggunakan PDO prepared statement.
- Menggunakan JOIN untuk menampilkan nama program studi.
- Menggunakan Bootstrap.
- Menambahkan konfirmasi JavaScript sebelum menghapus.
- Menambahkan pencarian mahasiswa berdasarkan nama atau NIM.

G. Pembahasan

Pada Acara 8 aplikasi dikembangkan dari aplikasi yang sebelumnya hanya menampilkan data menjadi aplikasi yang dapat melakukan proses CRUD. Pengelolaan data dilakukan menggunakan Repository sehingga proses database dipisahkan dari Controller.

Koneksi database menggunakan PDO melalui class Database. Class tersebut menggunakan pola singleton sehingga koneksi database dapat digunakan kembali oleh bagian aplikasi yang membutuhkan.

Prepared statement diterapkan pada query yang menerima input pengguna, seperti proses tambah, edit, hapus, dan pencarian. Penggunaan prepared statement membantu mencegah SQL Injection karena input pengguna dipisahkan dari perintah SQL.

JOIN digunakan pada data mahasiswa untuk menghubungkan tabel mahasiswa dengan tabel prodi sehingga nama program studi dapat ditampilkan. Hal yang sama diterapkan pada data mata kuliah.

Bootstrap digunakan untuk memperbaiki tampilan halaman agar lebih terstruktur. Selain itu, JavaScript digunakan untuk memberikan konfirmasi kepada pengguna sebelum proses penghapusan dilakukan.

Pada tugas mandiri, fitur pencarian mahasiswa berhasil ditambahkan. Pencarian dapat dilakukan berdasarkan NIM atau nama menggunakan LIKE dan prepared statement.

H. Kesimpulan

Acara 8 berhasil menerapkan CRUD pada data mahasiswa, program studi, dan mata kuliah menggunakan PHP, PDO, dan Repository.

Koneksi database berhasil dibuat menggunakan pola singleton. Prepared statement juga berhasil diterapkan untuk memproses input pengguna sehingga query database tidak langsung menggabungkan input dengan perintah SQL.

Selain itu, JOIN berhasil digunakan untuk menampilkan nama program studi pada data mahasiswa dan mata kuliah. Bootstrap digunakan untuk membuat tampilan lebih rapi, sedangkan JavaScript digunakan untuk memberikan konfirmasi sebelum menghapus data.

Fitur pencarian berdasarkan nama atau NIM juga berhasil ditambahkan menggunakan LIKE dengan prepared statement.